

SQLcl + VS Code: Easy Guide for Database Object Export

What is SQLcl?

SQLcl (SQL Command Line) is the modern version of SQL*Plus. It's a powerful tool for developers and APEX users.

SQL*Plus: Old, simple, text-only.

SQLcl: Modern, smart, developer-friendly.

✓ If you're a developer or an APEX user, **SQLcl** is your best choice.

Step 1: Create Your Local Project Folder

Choose a folder where you will store your code and exports.

Example:

D:\eAgentVersionControl\SONALI_ABS

Step 2: Open SQLcl in VS Code

Right-click your database connection in VS Code.

Click **"Open SQLcl"**.

A terminal will open below.

Change the directory to your project folder using the CD command.

Command:

```
CD D:\eAgentVersionControl\SONALI_ABS
```

Step 3: Initialize SQLcl Project

Create a SQLcl project for your schema(s).

Command:

```
project init -name SONALI_ABS -schemas EMOB
```

For multiple schemas:

```
project init -name SONALI_ABS -schemas EMOB,GUMS,BIOTPL,MEMIMG
```

Step 4: Export APEX Workspace / Applications

Export entire workspace:

```
apex export -expWorkspace -workspaceId 1400484024213076
```

Export all apps (single file):

```
apex export -workspaceid 1400484024213076
```

Export all apps (split files):

```
apex export -workspaceid 1400484024213076 -split
```

List apps:

```
apex export -list -workspaceid 1400484024213076
```

Export a single app:

```
apex export -applicationid 1234 -split
```

Export specific pages/LOVs:

```
apex export -applicationid 1234 -split -expComponents PAGE:3 PAGE:11  
LOV:23618973754424510000
```

Step 5: Export Database Objects

Export a whole schema:

```
project export -schemas EMOB
```

Export a single object:

```
project export -o FUNCTION_CALCULATE_SALARY  
project export -o VIEW_EMP_INFO
```

Step 6: Data Pump Export (Optional)

Create a directory:

```
CREATE OR REPLACE DIRECTORY EMOB_DUMP_DIR AS  
'D:\eAgentVersionControl\CITY_ABS\DUMP';
```

Export schema:

```
dp export -dumpfile my_schema.dmp -logfile my_schema.log
```

Export multiple schemas:

```
dp export -schemas HR,DEMO -dumpfile multi_schema.dmp -dir  
D:\eAgentVersionControl\CITY_ABS\DUMP
```

Final Goal: Source Version Controlling

The ultimate objective of this workflow is **source version control** for your database objects and APEX applications.

Step 7: Git Integration

Once your project folder is set up with exported objects, you can manage it with Git.

Initialize Git:

```
!git init
```

Add a remote repository:

```
!git remote add origin  
https://erainfotechbd@dev.azure.com/erainfotechbd/eAgent/_git/city_abs_app
```

Pull the latest code:

```
!git fetch origin  
!git pull origin main --rebase
```

Create a branch:

```
!git checkout -b miaze_dev  
!git push --set-upstream origin miaze_dev
```

Set your Git identity:

```
!git config --global user.name "Ayub Miaze"  
!git config --global user.email ayub@erainfotechbd.com
```

Recommended Daily Workflow

Pull latest changes:

```
!git pull origin main --rebase
```

Export updated objects:

```
project export -schemas EMOB
```

Commit changes:

```
git add .  
git commit -m "Updated EMOB schema objects"
```

Push to remote:

```
git push
```

Pro Tips

- ✓ Use **branches** for each new feature or bug fix (e.g., `miaze_dev`, `feature_fix_salary`).
- ✓ Use the `-split` option for APEX apps for **better version control** and easier tracking of individual component changes.
- ✓ Write **clear, descriptive commit messages**.
- ✓ Regularly sync your local code with the remote repository using `git pull --rebase`.